

Project Report

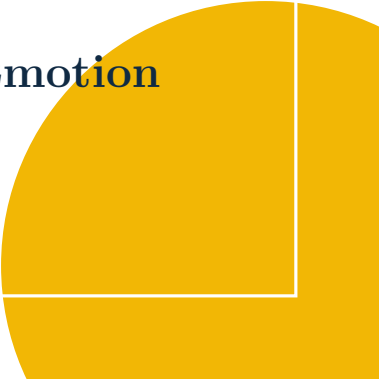
Project Domain: Artificial Intelligence, Deep Learning, and
Computer Vision

Development Language: Python

Submitted by: Manshi Verma

Date of Submission: 09 May 2026

**Deep Learning and Computer Vision Based Emotion
Classification**



Contents

1	Abstract	1
2	Introduction	1
3	Problem Statement	2
4	Objectives	2
5	Scope of the Project	3
6	Literature Review and Background Study	3
7	Proposed Methodology	4
7.1	Dataset Validation	4
7.2	Dataset Description	4
7.3	Image Preprocessing	5
7.4	Data Augmentation	5
7.5	Model Training	5
7.6	Model Evaluation	6
8	System Architecture and Workflow	6
9	Tools and Technologies Used	7
10	Implementation Details	7
10.1	Custom CNN Architecture	7
10.2	MobileNetV2 Transfer-Learning Architecture	7

10.3 Project Modules	8
10.4 Execution Commands	8
11 Results and Discussion	9
12 Advantages	10
13 Limitations	10
14 Future Scope	11
15 Ethical Considerations	11
16 Conclusion	12

1 Abstract

The Facial Emotion Recognition System is a deep-learning-based application developed to identify human emotions from facial images and live webcam video. The system classifies a detected facial expression into one of seven predefined emotion categories: angry, disgust, fear, happy, neutral, sad, and surprise. It provides a complete workflow that includes dataset validation, image preprocessing, model training, evaluation, single-image prediction, and real-time webcam-based inference.

The project implements two model approaches: a custom Convolutional Neural Network (CNN) and a MobileNetV2 transfer-learning architecture. TensorFlow and Keras are used for model development and training, while OpenCV is used for face detection, webcam access, and video frame processing. The dataset used in the project contains 35,817 facial images arranged into training and validation folders. To reduce the effect of class imbalance, the training pipeline supports calculated class weights, especially for underrepresented classes such as disgust. The best recorded fine-tuning experiment achieved a validation accuracy of approximately 65.63%, showing that the system can learn meaningful facial-expression features and perform practical emotion classification.

2 Introduction

Facial expressions are an important part of non-verbal human communication. They allow people to express feelings, reactions, and social signals without using words. Automatic recognition of facial expressions has become an active area of research in artificial intelligence and computer vision because it can support applications in human-computer interaction, education, healthcare, customer feedback analysis, intelligent monitoring, and assistive technologies.

The purpose of this project is to design and implement a practical facial emotion recognition system that can analyze a facial image and predict the emotion displayed by the person. In addition to image-based prediction, the system also performs real-time emotion recognition through a webcam. This makes the project useful not only as a model-training exercise but also as an applied computer vision system.

Deep learning is suitable for this task because facial expressions contain complex visual patterns, including changes around the eyes, mouth, eyebrows, and cheeks. A CNN can automatically learn these patterns from image data, while a transfer-learning model such as MobileNetV2 can reuse features learned from large-scale image datasets. This project

therefore explores both a compact custom CNN and a pretrained architecture to improve recognition performance.

3 Problem Statement

Human facial expressions vary significantly across individuals and environments. Factors such as lighting conditions, face orientation, camera quality, partial occlusion, and personal appearance can affect the visual features present in an image. In addition, some emotional expressions are visually similar; for example, fear and surprise may share widened eyes and open-mouth features. Manual interpretation of facial expressions is also subjective and time-consuming.

This project addresses the problem by training deep learning models to automatically extract visual features from facial images and classify them into seven emotion categories. The system further integrates the trained model with a webcam so that predictions can be generated in real time. The objective is not to determine a person's complete psychological condition, but to estimate the visible facial expression shown in an image or video frame.

4 Objectives

The main objectives of the Facial Emotion Recognition System are as follows:

1. To build a deep learning model for facial emotion classification.
2. To recognize seven facial emotion categories: angry, disgust, fear, happy, neutral, sad, and surprise.
3. To compare a custom CNN approach with a MobileNetV2 transfer-learning approach.
4. To apply calculated class weights for reducing the effect of class imbalance in the dataset.
5. To evaluate model performance using validation accuracy, validation loss, classification reports, and confusion matrices.
6. To support emotion prediction from individual image files.
7. To implement real-time facial emotion recognition using webcam input.
8. To provide a complete workflow from dataset validation to deployment-ready inference.

5 Scope of the Project

The scope of this project is limited to facial-expression classification using image and webcam input. The system accepts facial images from an organized dataset, an individual image file, or a live webcam stream. It detects faces, preprocesses the image according to the selected model architecture, and predicts the most likely emotion class.

The project includes training, evaluation, prediction, and real-time inference modules. It does not attempt to infer a person's full emotional state, personality, intention, or mental health condition. The output should be understood as an estimate based only on visible facial features. The accuracy of the system depends on the quality, diversity, and balance of the training dataset, as well as environmental conditions during inference.

6 Literature Review and Background Study

Facial emotion recognition is a subfield of computer vision that focuses on identifying human emotions from facial expressions. Early approaches often depended on handcrafted features, such as edge descriptors, texture features, and landmark-based measurements. These methods required careful feature engineering and were sensitive to variations in pose, lighting, and image quality.

With the growth of deep learning, CNNs have become widely used for image classification tasks. CNNs are effective because they learn hierarchical visual features directly from image pixels. Lower layers learn simple patterns such as edges and corners, while deeper layers learn more complex structures such as facial components and expression-related patterns. This makes CNNs suitable for facial-expression recognition, where subtle differences in facial regions can represent different emotions.

Transfer learning is another important technique in modern computer vision. Instead of training a model from the beginning, a pretrained model can be used as a feature extractor. MobileNetV2 is a lightweight CNN architecture designed for efficient image recognition. It uses inverted residuals and linear bottlenecks to reduce computational cost while maintaining strong feature extraction capability. In this project, MobileNetV2 is used as a transfer-learning model to compare with the custom CNN approach.

The dataset structure used in this project is consistent with the FER-2013 facial-expression dataset format. FER-2013 is a commonly used benchmark dataset for facial expression recognition and contains grayscale face images categorized into standard emotion classes. Although such datasets are useful for training and evaluation, they may contain class im-

balance and may not represent all real-world variations. Therefore, careful preprocessing, evaluation, and ethical use are important for building a reliable system.

7 Proposed Methodology

The proposed methodology follows a structured machine learning pipeline. It begins with dataset validation and preprocessing, continues with model training and evaluation, and ends with image-based and real-time inference.

7.1 Dataset Validation

Before training begins, the system verifies that the dataset contains the required training and validation directories. Each directory must include matching folders for the seven emotion classes. This step prevents errors caused by missing folders, incorrect class names, or incomplete dataset organization.

7.2 Dataset Description

The dataset contains a total of 35,817 facial images. The images are divided into training and validation subsets. Table 1 presents the emotion classes used by the system, and Table 2 shows the dataset distribution.

Table 1: Emotion classes used in the system

Class Number	Emotion
1	Angry
2	Disgust
3	Fear
4	Happy
5	Neutral
6	Sad
7	Surprise

Table 2: Dataset distribution across emotion classes

Emotion	Training Images	Validation Images	Total Images
Angry	3,995	888	4,883
Disgust	436	111	547
Fear	4,097	1,024	5,121
Happy	7,215	1,774	8,989
Neutral	4,965	1,233	6,198
Sad	4,830	1,247	6,077
Surprise	3,171	831	4,002
Total	28,709	7,108	35,817

The distribution shows class imbalance, especially for the disgust class, which has far fewer images than the other categories. To address this issue, the training process can calculate balanced class weights and apply them during model fitting.

7.3 Image Preprocessing

The preprocessing strategy depends on the selected architecture. The custom CNN uses 48 x 48 grayscale images, which are suitable for compact facial-expression datasets. The MobileNetV2 model uses 224 x 224 RGB images because this is the expected input format for the pretrained architecture. Input images are resized automatically, and pixel values are rescaled inside the model.

7.4 Data Augmentation

Data augmentation is applied during training to improve the model's ability to generalize to unseen images. The project uses augmentation operations such as horizontal flipping, small random rotations, and random zooming. These transformations expose the model to realistic variations and help reduce overfitting.

7.5 Model Training

The models are trained using the Adam optimizer and categorical cross-entropy loss. Accuracy and loss are monitored for both the training and validation sets during each epoch. The training pipeline includes model checkpointing, early stopping, learning-rate reduction, CSV logging, optional class weighting, and support for resuming training from a saved checkpoint.

7.6 Model Evaluation

After training, the model is evaluated on the validation dataset. The evaluation process generates validation accuracy, validation loss, a classification report, and a confusion matrix. The classification report provides precision, recall, and F1-score for each emotion class, while the confusion matrix shows which classes are correctly classified and which classes are commonly confused.

8 System Architecture and Workflow

The system follows a modular architecture in which each module performs a specific responsibility. The general workflow is shown in Figure 1.

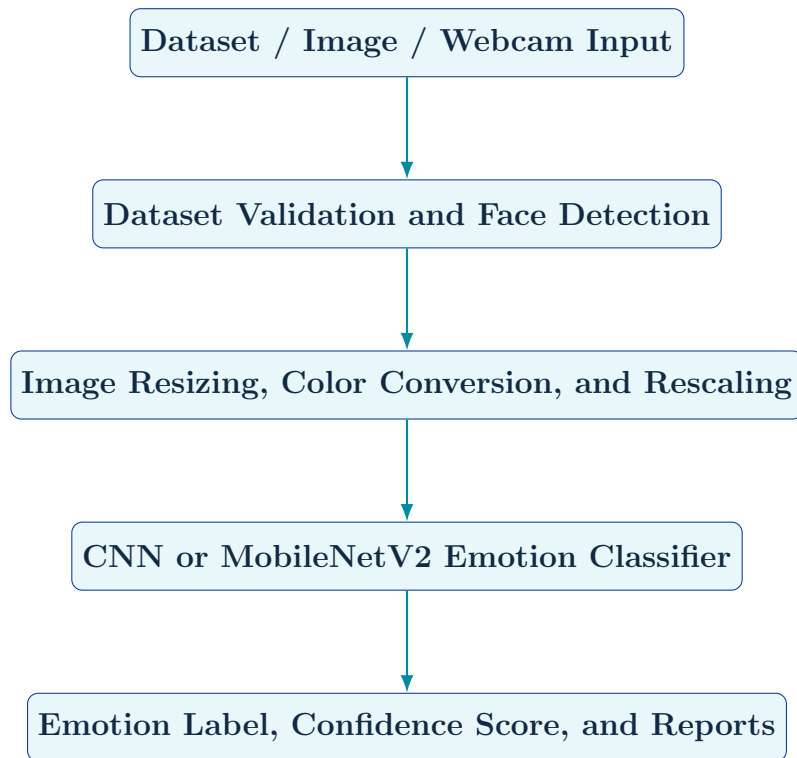


Figure 1: Suggested system workflow for facial emotion recognition

For real-time inference, OpenCV captures frames from the webcam. A Haar Cascade classifier detects faces in each frame. Every detected face is cropped, resized, and passed to the trained model. The system averages predictions across recent frames to reduce sudden label changes. If the prediction confidence is below the configured threshold, the result is displayed as **uncertain**.

9 Tools and Technologies Used

Table 3 summarizes the main technologies used in the project.

Table 3: Tools and technologies used

Technology	Purpose
Python	Core programming language
TensorFlow and Keras	Model creation, training, evaluation, and inference
OpenCV	Webcam access, face detection, and video display
NumPy	Numerical operations and prediction processing
Pillow	Image loading and preprocessing
Scikit-learn	Class weights, classification report, and confusion matrix
Matplotlib and Seaborn	Confusion matrix visualization
Haar Cascade Classifier	Real-time frontal-face detection

10 Implementation Details

10.1 Custom CNN Architecture

The custom CNN is designed for 48×48 grayscale facial images. It contains three convolutional feature-extraction blocks, batch normalization layers, max-pooling layers, dropout layers, global average pooling, and a dense softmax output layer. This architecture is compact and suitable for low-resolution facial-expression images. Dropout and batch normalization help improve training stability and reduce overfitting.

10.2 MobileNetV2 Transfer-Learning Architecture

The second architecture uses MobileNetV2 as a feature-extraction base. It supports ImageNet pretrained weights and selective fine-tuning of the final layers of the base model. The output of MobileNetV2 is connected to global average pooling, dense layers, batch normalization, dropout layers, and a seven-class softmax classifier. Transfer learning helps the system reuse general visual features learned from a large image dataset.

10.3 Project Modules

The implementation is organized into multiple files, as shown in Table 4.

Table 4: Major project modules and responsibilities

File or Directory	Responsibility
<code>code/model.py</code>	Dataset loading, model training, evaluation, and image prediction
<code>code/camera_emotion.py</code>	Real-time webcam emotion detection
<code>code/check_dataset_counts.py</code>	Counts images in each dataset class
<code>emotion_model.keras</code>	Saved trained Keras model
<code>emotion_labels.json</code>	Ordered list of emotion labels
<code>reports/</code>	Training histories and confusion matrix images

10.4 Execution Commands

The project can be executed using command-line instructions. The required packages are installed using:

```
pip install -r requirements.txt
```

The default custom CNN can be trained using:

```
python code/model.py --mode train
```

The MobileNetV2 model can be trained using:

```
python code/model.py --mode train --architecture mobilenetv2
```

A saved model can be evaluated using:

```
python code/model.py --mode evaluate
```

Emotion prediction from a single image can be performed using:

```
python code/model.py --mode predict --image path\to\image.jpg
```

Real-time webcam emotion recognition can be started using:

```
python code/camera_emotion.py
```

The webcam window can be closed by pressing the **q** key.

11 Results and Discussion

The project contains recorded histories for multiple experiments, including smoke tests, MobileNetV2 training, weighted training, and fine-tuning. The best recorded fine-tuning result is shown in Table 5.

Table 5: Best recorded fine-tuning result

Metric	Recorded Value
Training Accuracy	70.11%
Validation Accuracy	65.63%
Training Loss	0.8052
Learning Rate	0.0001
Recorded Epoch Index	4

The validation accuracy of approximately 65.63% indicates that the model learned useful expression-related features from the dataset. The training accuracy of 70.11% is moderately higher than the validation accuracy, which suggests that the model generalizes reasonably but may still benefit from additional regularization, improved augmentation, and more balanced data.

The confusion matrix generated by the evaluation pipeline is useful for identifying class-level errors. For example, visually similar expressions such as fear and surprise may be confused because both can involve widened eyes and an open mouth. The disgust class may also be difficult to classify reliably because it contains far fewer training images than the other categories. These observations show why class balancing, additional data collection, and improved model tuning are important for future work.

Figure suggestion: A confusion matrix image from the **reports/** directory may be inserted here to visually show correct and incorrect predictions across all seven emotion classes.

12 Advantages

The major advantages of the proposed system are as follows:

1. It supports both custom CNN and transfer-learning approaches.
2. It performs emotion prediction from both individual images and live webcam video.
3. It automatically handles grayscale and RGB input requirements depending on the model architecture.
4. It uses prediction smoothing to improve the stability of real-time webcam output.
5. It includes safeguards against overfitting, such as dropout, early stopping, and learning-rate reduction.
6. It generates evaluation reports and confusion matrices for performance analysis.
7. It supports resuming model training from a saved checkpoint.

13 Limitations

The system also has some limitations:

1. Prediction accuracy may decrease under poor lighting conditions or unusual face angles.
2. Similar facial expressions, such as fear and surprise, can be difficult to distinguish.
3. The dataset is imbalanced, with significantly fewer images for the disgust class.
4. Haar Cascade face detection may not reliably detect partially visible, rotated, or heavily occluded faces.
5. Facial expression alone cannot always represent a person's actual emotional condition.
6. Model performance may vary across different individuals, devices, and camera qualities.

14 Future Scope

The project can be improved in several ways:

1. A more robust face detector, such as MediaPipe or a deep-learning-based detector, can be used for improved face localization.
2. Systematic hyperparameter tuning can be performed to optimize learning rate, batch size, dropout rate, and fine-tuning depth.
3. More diverse and balanced facial-expression images can be added to improve generalization.
4. A graphical user interface or web application can be developed for easier user interaction.
5. The trained model can be deployed on mobile or edge devices for real-time lightweight inference.
6. Emotion analytics and prediction-history visualization can be added for better interpretation of results.
7. Fairness and accuracy can be evaluated across different demographic groups to support responsible use.

15 Ethical Considerations

Facial emotion recognition systems must be used responsibly because they process sensitive biometric information. Users should provide informed consent before their facial images or webcam streams are analyzed. Any collected images, video frames, or prediction logs should be stored securely and should not be shared without permission.

The predictions generated by the system should be treated as estimates of visible facial expressions, not as definite conclusions about a person's internal emotional or mental state. The system should not be used for high-impact decisions in areas such as employment, health-care diagnosis, legal judgment, or academic evaluation without proper validation, human oversight, and ethical approval.

16 Conclusion

The Facial Emotion Recognition System demonstrates the practical use of deep learning and computer vision for classifying facial expressions. It provides an end-to-end workflow that includes dataset validation, preprocessing, model training, evaluation, image prediction, and real-time webcam inference. The system supports both a custom CNN and a MobileNetV2 transfer-learning model, allowing comparison between a compact architecture and a pretrained feature-extraction approach.

The project achieved a best recorded validation accuracy of approximately 65.63% during fine-tuning. This result shows that the model can learn meaningful facial-expression features, although further improvements are possible. With more balanced training data, stronger face detection, systematic hyperparameter tuning, and careful ethical deployment, the system can become more reliable and useful for practical applications in human-computer interaction and intelligent visual analysis.

References

- [1] TensorFlow. *TensorFlow and Keras Documentation*. Available at: <https://www.tensorflow.org/>.
- [2] OpenCV. *OpenCV Documentation*. Available at: <https://docs.opencv.org/>.
- [3] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L. C. “MobileNetV2: Inverted Residuals and Linear Bottlenecks.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [4] Goodfellow, I. J., Erhan, D., Carrier, P. L., Courville, A., Mirza, M., Hamner, B., Cukierski, W., Tang, Y., Thaler, D., Lee, D. H., Zhou, Y., Ramaiah, C., Feng, F., Li, R., Wang, X., Athanasakis, D., Shawe-Taylor, J., Milakov, M., Park, J., Ionescu, R., Popescu, M., Grozea, C., Bergstra, J., Xie, J., Romaszko, L., Xu, B., Chuang, Z., and Bengio, Y. “Challenges in Representation Learning: A Report on Three Machine Learning Contests.” *Neural Information Processing*, 2013.
- [5] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.